
WatchTower: A Transformer-Based Open-Set Network Anomaly Detection System for Edge Devices

Usman Ahad¹ Umer Irfan¹

Abstract

Network anomaly detection on edge devices presents a unique challenge: models must be lightweight enough for constrained hardware yet robust enough to detect evolving, open-set attack patterns. Traditional statistical methods fail against sophisticated attacks, while current Deep Learning solutions, such as LSTM-Autoencoders, struggle with long-term temporal dependencies and "catastrophic forgetting" in dynamic environments. In this work, we propose *WatchTower*, a Transformer-based anomaly detection pipeline optimized for the Raspberry Pi 5. By integrating a state of the art open-set recognition technique (PROSER) and Federated Learning (Fed-Prox), our model learns to distinguish benign traffic from anomalous packets without overfitting to local network data traffic. We further deploy an end-to-edge optimization stack, utilizing Static Post-Training Quantization (PTQ) and hardware-aware TVM auto-tuning to achieve a real-time latency over unoptimized baselines. Our approach achieves **93.66% accuracy**, outperforming standard LSTM baselines by over 19%, demonstrating that high-performance, real-time intrusion detection is viable on commodity edge hardware. The GitHub source files for this project have been made public [here](#).

1. Introduction

Network anomaly detection has been an ever evolving problem, not because it is tough to address, but because the dynamic nature of open set recognition allows for a multitude of techniques to be implemented with respect to compute, memory, and latency constraints.

Current SOTA solutions rely on storing the data and then computing it after a set time. This leads to increasing storage requirements, not to mention that breaches and attacks are detected post fact.

In recent times, instead of trying to use traditional statistical

metrics (which are only effective against floods), neural networks are being employed at an increasing rate.

2. Background and Related Work

Early neural techniques revolved around clustering and segmentation, using hand made or learned pattern recognizers and feature mappings. 1D CNNs have been increasingly utilized to learn normal patterns, to then distinguish between seen and unseen patterns.

This introduces a key problem of modeling networks. With time, demand, and real world random events the network patterns shift so dramatically, that a simple model must be retrained to keep its labels accurate.

Many models try to solve this by utilizing reconstruction scores as a metric of anomaly. This works to an extent however is very unreliable because modern day network traffic is becoming increasingly diverse. A YouTube program buffering 100 seconds worth of footage is indistinguishable from a UDP DOS attack in terms of network features.

Thus instead of using simple Autoencoders (Mirsky et al., 2018) newer approaches try to incorporate LSTM heads to extract temporal dependencies and try to reconstruct temporal features.

This LSTM based reconstruction has been the state of the art solution to date (Elsayed et al., 2020).

However LSTMs struggle with correlating across longer time periods because of their construction. This is the gap we aim to exploit. Transformers (ie: self attention) can catch more intricate features between all the packets, and maintain a better context across moving windows of packet features.

3. Methodology

In this section, we introduce all the key techniques involved in detecting anomalous packets using a transformer based discriminator. We aim to provide a comprehensive approach to tackling this problem, from dataset preparation, model architecture, a SOTA open-set recognition method, employment in a federated setting, and finally, optimizations to

maximize our model’s performance on edge devices.

Dataset Preparation

We used the Intrusion detection evaluation dataset (CIC-IDS2017) (Sharafaldin et al., 2017) (specifically, the Monday, Tuesday, and Wednesday portions of the dataset) to train our model to differentiate the benign data from the anomalous packets. We further incorporated custom data that was harnessed using Wireshark to finetune the training set and ensure the model does not overfit onto the sole network within the CIC-IDS2017 dataset. The following features were used from each packet to train the model:

- `inter_arrival_time`: The time between the arrival of the current packet and the previous packet in the flow.
- `protocol_number`: The numeric identifier for the transport layer protocol used (e.g., 6 for TCP, 7 for UDP).
- `ip_total_length`: The total size of the IP packet in bytes.
- `ip_header_length`: The length of the IP header itself.
- `payload_length`: The size of the packet data.
- `ttl`: The Time-To-Live value indicating the maximum number of hops the packet can traverse before being discarded.
- `byte0-byte15`: The first 16 bytes of the packet payload, often used for deep packet inspection or signature matching. These will be used to calculate the payload related fields.
- `df_flag`: The "Don't Fragment" bit, which indicates whether the packet is allowed to be fragmented by routers.
- `mf_flag`: The "More Fragments" bit, set to 1 if there are subsequent fragments following the current one.
- `fragment_offset`: The position of the current fragment’s data relative to the beginning of the original unfragmented packet payload.
- `payload_entropy`: A statistical measure of the randomness of the payload data, often used to detect encryption or compression.
- `payload_mean`: The arithmetic mean of the byte values contained within the packet payload.
- `payload_std`: The standard deviation of the byte values in the payload, indicating the spread of data distribution.

Finally, evaluation of the model was conducted on real-time network traffic with 5 devices connected to a Raspberry Pi 5 (which served as the router), where the [Low-Orbit Ion Cannon](#) was used to simulate DoS attacks to see if the implementation worked as intended.

Model Architecture

The model incorporated a number of different components to maximise the relationship between the packets and windows in order for it to detect any semblance of anomalies between packets. Key components include:

The model incorporated a number of different components to maximise the relationship between the packets and windows in order for it to detect any semblance of anomalies between packets. Key components include:

- **Feature Embedding Layer:** Our model projects each of the 28 raw packet features independently into a learned embedding space before concatenation. This preserves the semantic distinction between each of the features (e.g., distinct IP flags vs. payload entropy) while projecting them into a unified d_{model} dimension (128 in our implementation) suitable for attention mechanisms.
- **Memory Token Layer:** Inspired by recent advances in Recurrent Transformers, we introduce learnable memory tokens that are concatenated to the input sequence. A custom *Memory Attention* mechanism updates these tokens using the current window’s data. Crucially, a **Gating Mechanism** (via a Sigmoid activation) is employed to selectively update the memory:

$$M_{new} = M_{old} \cdot (1 - g) + M_{update} \cdot g \quad (1)$$

This allows the model to retain long-term context across separate windows. This is a transformer style attempt to address the "catastrophic forgetting" often seen in standard sliding-window Transformers, to retain global context via the *attention mechanism*.

- **Positional Encoding:** Since the Transformer architecture contains no inherent structure for sequence order, a learnable positional embedding is added to the packet sequence. This ensures the model understands the order of the packets as they come in real-time, which is essential for identifying sequential anomalies like TCP handshake violations as seen in TCP SYN floods.
- **Transformer Encoder Blocks:** The main bulk of the transformer model consists of stacked Multi-Headed

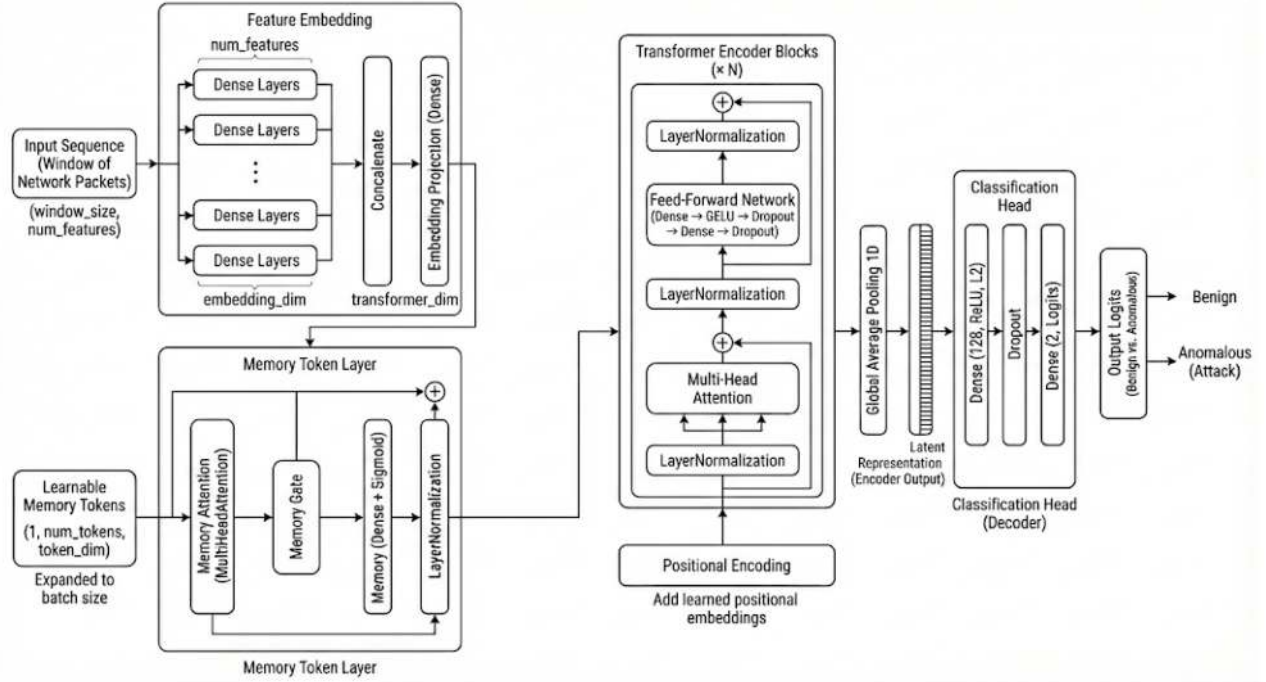


Figure 1. Model Architecture

Self-Attention (MHSA) blocks and GeLU activations. These allow every packet in the window to attend to every other packet, effectively capturing complex, non-linear intra-window dependencies that simple statistical models miss.

- **Latent Representation & Classification Head:** The output of the transformer blocks is condensed via Global Average Pooling into a single latent vector representing the entire window. This latent representation is passed through a dense classification head with L2 regularization (0.001) to prevent overfitting. We deliberately expose this latent vector (prior to the final logits) to facilitate the Open-Set Recognition (PROSER) techniques described in the following section.

The entire model architecture can be visualised in the Figure 1.

PROSER

A critical limitation of standard discriminative models in network security is their tendency to be overconfident when encountering Out-of-Distribution (OOD) data, such as zero-day attacks that were not present in the training set. Since the model has not been exposed to zero-day attacks during training, it may incorrectly classify the attack as benign. To address this "Open Set" problem, we implemented the PROSER (Placeholder for Open-Set Recognition) algo-

rithm.

The core intuition is to simulate "unknown" attacks during training by generating synthetic outliers. Instead of mixing raw input packets (which might destroy protocol syntax), we perform *Manifold Mixup* in the latent space produced by our Transformer Encoder from our Benign packet data in a technique known as **PROSER** (Zhou et al., 2021).

The procedure, as implemented in our training loop, operates in three steps:

1. **Latent Extraction:** We first pass a batch of real benign traffic x through the encoder to obtain their latent representations $z = \text{Encoder}(x)$.
2. **Manifold Mixing:** We generate synthetic latent points $\tilde{z}$ by interpolating between two random latent vectors z_i and z_j from the current batch:

$$\tilde{z} = \beta \cdot z_i + (1 - \beta) \cdot z_j \quad (2)$$

where $\beta \sim U(0, 1)$ is a random mixing coefficient. These synthetic points represent data that lies "between" the established clusters of known network patterns, i.e., this can be interpreted as anomalous — effectively producing anomalous latents from normal benign data.

3. **Anomaly Enforcement:** These mixed representations

$\tilde{z}$ are passed through the Decoder and explicitly labeled as anomalies (Class 1).

By penalizing the model for classifying these interpolated points as benign and rewarding them for being classified as anomalous, we effectively compress the decision boundary tightly around the manifold of normal traffic. This ensures that any network pattern falling into the empty space of the latent distribution—characteristic of novel attacks—is rejected as an anomaly rather than being confidently misclassified as normal. This coupled with attack data being present in the training data helps the model be exposed to a variety of attack latent representations that it can use to learn, and, further, detect various types of attacks during inference.

FedProx

For network anomaly detection, client drift between all the router (due to network traffic being fundamentally different across devices and throughout the day) is a massive issue and could lead to catastrophic forgetting. As the model adapts to the network traffic, it learns new dependencies and the relationship between packets in the incoming windows of packets and could forget its initial training dataset. To prevent this, a regulariser term is added to the simple Federated Averaging aggregation method, which is called FedProx (Li et al., 2020). FedProx (Li et al., 2020) extends FedAvg by adding a proximal term to each client’s local loss to reduce deviation from the global model:

$$\mathcal{L}_{\text{FedProx}} = \mathcal{L}_{\text{local}} + \frac{\mu}{2} \|\theta_{\text{local}} - \theta_g^t\|^2, \quad (3)$$

where θ_g^t is the global model at the start of the round, θ_{local} are the local parameters, and μ controls the regularization strength. When $\mu = 0$, this reduces to FedAvg. The proximal term stabilizes training on non-IID data by limiting variance between client updates.

Optimisations

For maximum efficiency and performance on Raspberry Pi 5, incorporating a number of different optimisations was imperative.

Static Post-Training Quantisation was done into FP-32 and into INT-8 was done on our model using TFLite. These models were deployed on the edge device and benchmarked for latency and accuracy.

TVM is a deep learning compiler stack that helps to optimize models for various hardware platforms, improving their performance and efficiency. For on-edge deployment, it is extremely helpful in reducing latency, maximising throughput and reducing energy requirements for model

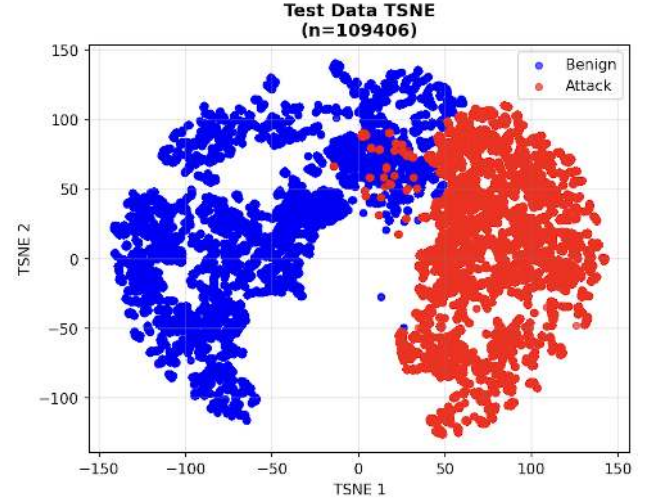


Figure 2. TSNE plot of the Test Data Samples after Complete training of the Model. It shows clear separation between the benign and anomalous packet data.

inference. To deploy our transformer-based detector on the Raspberry Pi 5, we utilized this Apache TVM compiler stack to perform hardware-aware optimization. We first took the quantized TFLite model into TVM’s high-level Relay IR to simplify the computation graph. To maximize inference efficiency, we implemented a custom auto-tuning pipeline specifically for the BCM2712 SoC:

- **Target Specification:** We explicitly compiled the model for the `llvm -mcpu=cortex-a76` target, enabling the use of ARMv8.2-A specific vector instructions. This was specifically for the RPi5.
- **Task Extraction & Auto-Tuning:** We extracted tunable subgraphs (such as dense attention layers) and employed a *Random Tuner* to empirically search through 1,000 configuration trials per task. This process identified the optimal loop tiling and memory layout strategies for the device’s specific cache hierarchy.
- **Benchmarking:** We validated these gains by comparing the mean inference latency of the fully tuned kernel (Opt Level 3) against an unoptimized baseline to quantify the performance uplift.

4. Evaluation and Results

In this section, we evaluate the efficacy of our Transformer-based approach against the established LSTM-Autoencoder baseline. We further analyze the interpretability of our model’s latent space and quantify the performance gains achieved through our edge-optimization pipeline on the Raspberry Pi 5.

Comparative Analysis

We benchmarked our model against a standard LSTM-Autoencoder (LSTM-AE) trained on a similar network packet dataset. As shown in Table 1, our Transformer-based architecture significantly outperforms the baseline across all key metrics, albeit our model requires significantly more compute as the LSTM-AE is approximately 300k parameters, whereas ours is 1M parameters.

While the LSTM-AE struggles to capture long-term dependencies—peaking at an accuracy of 74.1%, our model achieves a top-1 accuracy of 93.66%. Notably, the F1-Score, which represents the harmonic mean of precision and recall, saw an improvement of over 10 percentage points (82.5% to 92.83%), indicating a far better balance between false positives and false negatives.

Table 1. Performance Comparison: LSTM-AE Baseline vs. Our Transformer Model

Model	Accuracy	Precision	Recall	F1-Score
LSTM-AE (Baseline)	74.10%	71.11%	98.30%	82.50%
Ours (Transformer)	93.66%	93.00%	91.66%	92.83%

Latent Space & Real-Time Detection

To understand the model’s decision boundary, we visualized the high-dimensional latent representations using t-SNE (t-Distributed Stochastic Neighbor Embedding). As illustrated in Figure 2, the model learns a distinct manifold separation between Benign traffic (Blue) and Attack patterns (Red). This clear clustering confirms that our PROSER-based training strategy effectively forces anomalies into a separate region of the feature space, rather than simply memorizing training examples.

This separation translates directly to real-time inference capabilities. During our live deployment test using the LOIC tool to simulate a DoS attack, the model’s anomaly probability score remained consistently low (< 0.3) during normal activity. However, as the attack commenced, the confidence sharply spiked (> 0.9), going well above the decision threshold (0.5) instantaneously (Figure 3). This demonstrates the model’s ability to generalize to live, unseen traffic flows with minimal latency.

Edge Optimization Performance

Deploying complex Transformers on edge hardware requires rigorous optimization. We evaluated two distinct acceleration strategies: Static Post-Training Quantization (PTQ) via TFLite, and Hardware-Aware Auto-Tuning via TVM.

TFLite Quantization Converting the model weights from FP32 to INT8 resulted in a 67.5% reduction in model size

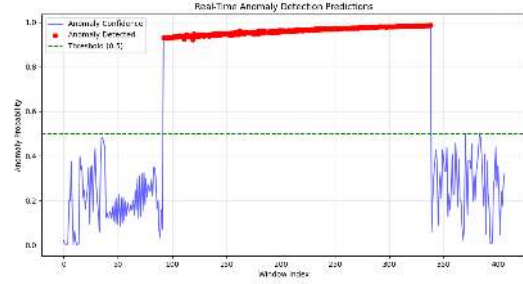


Figure 3. This was the result of the real-time INT8 model deployed on RPi5 after TVM optimisation. The DoS attack (starting at around window index 95) can clearly be identified

(4.63 MB to 1.51 MB). This compression reduced memory bandwidth pressure, improving inference latency by approximately $1.4\times$ (from 9.9ms to 6.1ms) with only a negligible drop in accuracy (2.33%).

TVM Auto-Tuning To push performance further, our TVM optimization pipeline (targeting the cortex-a76 instruction set) provided the most significant gains. By searching for optimal loop tiling and vectorization parameters over 1,000 trials, the TVM-compiled kernel reduced inference latency to just **3.82 ms**. This represents a total speedup of $2.6\times$ over the unoptimized baseline, making the system capable of processing over 260 packets per second on a single core.

Table 2. Ablation Study of Edge Optimizations on Raspberry Pi 5

Optimization Stage	Latency	Speedup	Model Size
Baseline (TF Float32)	9.91 ms	1.0x	4.63 MB
TFLite (INT8 PTQ)	6.09 ms	1.6x	1.51 MB
TVM (Auto-Tuned)	3.82 ms	2.6x	1.51 MB

5. Future Works, Discussion and Limitations

Our experimentation demonstrates a transformer-based pipeline that meets and exceeds LSTM based solutions in our preliminary testing. Our quantized INT8 model achieved $1.40\times$ speedup with only 2.33% accuracy drop, making our 1M parameter model more than suitable for deployment on edge devices.

However there exist limitations such that the CIC-IDS2017 dataset may not exhibit significant temporal shifts. Additionally the LSTM solution was benchmarked on a different dataset. This does not present a significant problem in our preliminary testing since the datasets were similarly structured, however deeper variations in traffic patterns exist. It is also worth mentioning that the LSTM-AE utilized more and different features than ours, so a apples to apples comparisons remains to be explored.

Finally, evaluation at realistic network scales (30 to 100 devices) with legitimate high-bandwidth (>500 mbps) scenarios and stress testing for false positive rates remain necessary. Our exploration evaluated at 100mbps rates with 3 to 8 connected devices.

6. Conclusion

We have presented WatchTower, a transformer-based open-set network anomaly detection system designed for deployment on edge devices. Our approach aims to exploit self-attention mechanisms to capture long range temporal dependencies in network traffic, addresses computational constraints through quantization and optimization, and incorporates federated learning to enable collaborative threat detection across distributed edge networks. Initial results hint towards performance exceeding LSTM-based baselines. This work contributes to an emerging direction in edge cybersecurity where edge devices serve as a first layer of defense against network intruders.

References

- Elsayed, M. S., Le-Khac, N.-A., Dev, S., and Jurcut, A. D. Network anomaly detection using LSTM based autoencoder. In *Proceedings of the 16th ACM Symposium on QoS and Security for Wireless and Mobile Networks*, Q2SWINet '20, pp. 37–45, Alicante, Spain, November 2020. ACM. doi: 10.1145/3416013.3426457. URL <https://doi.org/10.1145/3416013.3426457>.
- Li, T., Sahu, A., Zaheer, M., Sanjabi, M., Talwalkar, A., and Smith, V. Federated optimization in heterogeneous networks. In *Proceedings of the 3rd Conference on Machine Learning and Systems (MLSys)*, 2020. URL <https://arxiv.org/abs/1812.06127>. Proposed Fed-Prox, addressing stability under heterogeneity.
- Mirsky, Y., Doitshman, T., Elovici, Y., and Shabtai, A. Kitsune: An ensemble of autoencoders for online network intrusion detection, 2018. URL <https://arxiv.org/abs/1802.09089>.
- Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A. Intrusion detection evaluation dataset (cic-ids2017), 2017. URL <https://www.unb.ca/cic/datasets/ids-2017.html>. Accessed: December 25, 2025.
- Zhou, D.-W., Ye, H.-J., and Zhan, D.-C. Learning place-holders for open-set recognition, 2021. URL <https://arxiv.org/abs/2103.15086>.